AWS VPC Peering - Connect Two VPCs

EWB Cloud Labs - Student Project 01 / Oct 2026

Name	Naga Ganesh Eswar Gangisetty
Batch	AWS Internship Batch 1.1
Project	VPC Peering with EC2, route tables, security groups and a bastion host
AWS Region	Asia Pacific (Mumbai) - ap-south-1
AWS Account ID	357601816129
Completion date	21 October 2026

1. Architecture and IP plan

Two VPCs in one account and Region. VPC A (10.10.0.0/16) holds a public subnet with a bastion host and a private subnet with application server App-A. VPC B (10.20.0.0/16) holds only a private subnet with application server App-B. A VPC peering connection joins the two VPCs so the app servers talk over private IPv4 addresses. The bastion is the only internet entry point (SSH), reached through VPC A's internet gateway. App-A and App-B have no public IP addresses.

Laptop --SSH/22--> Internet gateway --> ewb-bastion 10.10.1.10 (public subnet A) --SSH--> App-A 10.10.2.10 (private subnet A); Bastion --SSH over peering--> App-B 10.20.1.10 (private subnet B). Project traffic: App-A <--> App-B over the peering connection.

Resource	VPC / Subnet	IPv4 CIDR / address
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-a-public	VPC A / public subnet	10.10.1.0/24
ewb-a-private	VPC A / private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B / private subnet	10.20.1.0/24
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4 13.201.3.177
ewb-app-a	ewb-a-private	10.10.2.10 (no public IP)
ewb-app-b	ewb-b-private	10.20.1.10 (no public IP)

The two VPC CIDRs do not overlap (10.10 vs 10.20), which is a hard requirement for VPC peering. Each /24 subnet sits inside its own VPC range.

2. Resource inventory (actual IDs)

Resource	ID
VPC A (ewb-vpc-a)	vpc-00062899eb4d741e5
VPC B (ewb-vpc-b)	vpc-03d4b2e09c334f54c
Subnet ewb-a-public	subnet-09a04db4102f80299
Subnet ewb-a-private	subnet-00dd0c2263d449df7
Subnet ewb-b-private	subnet-0871e056c5a76b280
Internet gateway ewb-igw-a	igw-0e9f0dff88c864324
Route table ewb-rt-a-public	rtb-099ba0a5e532a7416
Route table ewb-rt-a-private	rtb-069d673de4ca1984d
Route table ewb-rt-b-private	rtb-08e30dc5076bb1ce1
Security group ewb-sg-bastion	sg-0657c2d957a398c60
Security group ewb-sg-app-a	sg-004ad6f60fccbafc4
Security group ewb-sg-app-b	sg-0253eeae519fbd397
Peering connection ewb-a-b-peer	pcx-0a9e746e48f246384
Instance ewb-bastion	i-07fe0033236692014 (t3.micro)
Instance ewb-app-a	i-0a3e6d009347c0bb7 (t3.micro)
Instance ewb-app-b	i-06bff114db7029409 (t3.micro)
Key pair	ewb-lab (RSA)

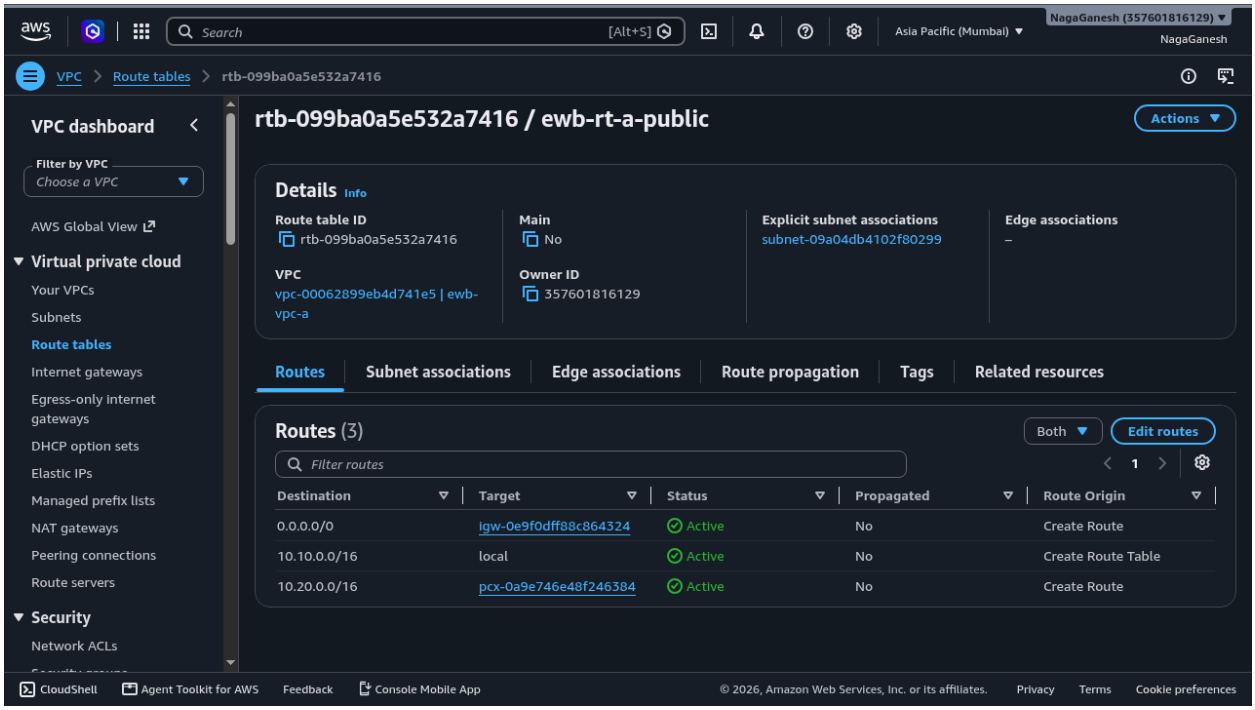
Resource	ID
AMI	ami-04394a3a96907e9ec (Amazon Linux 2023, ap-south-1)

All resources carried the tags Project = EWB-VPC-Peering and Student = NagaGanesh.

3. Subnets, internet gateway and route tables

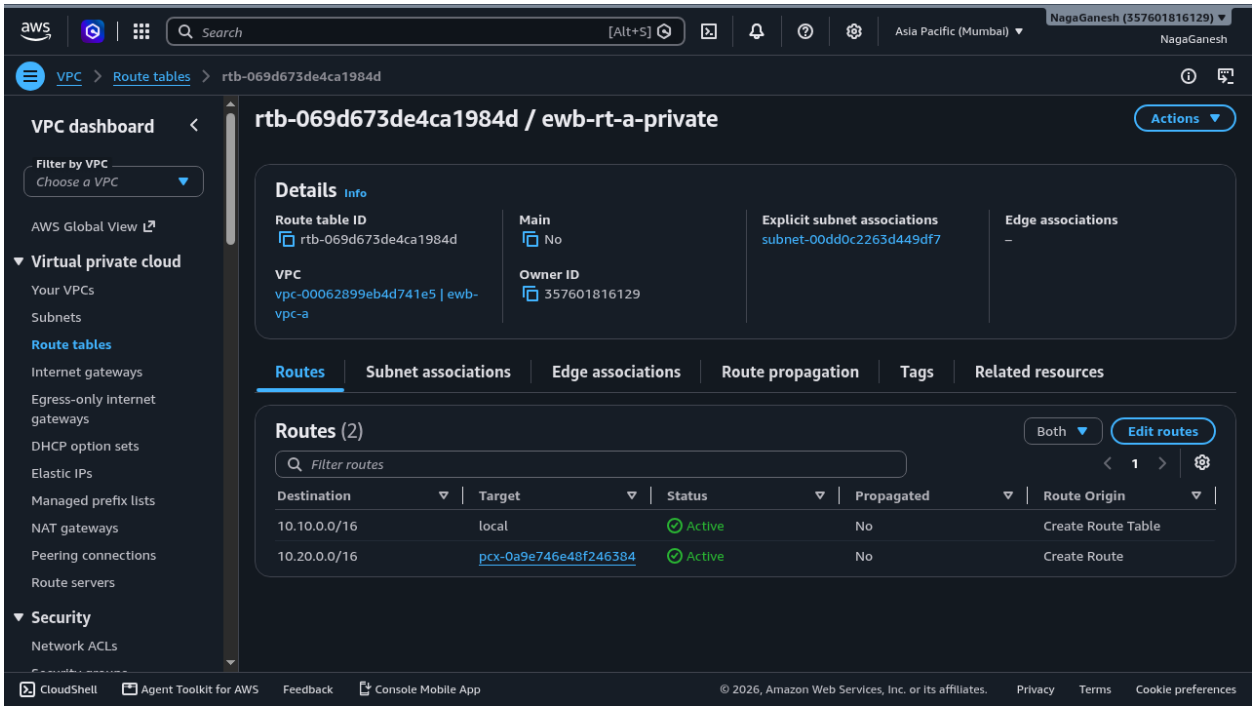
Both subnets of VPC A and the single subnet of VPC B were created in ap-south-1a with public IPv4 auto-assign disabled on the private subnets. The internet gateway ewb-igw-a was attached to VPC A only. Each route table was explicitly associated with its subnet; the private tables kept only their local route until peering was added.

ewb-rt-a-public (associated: ewb-a-public)



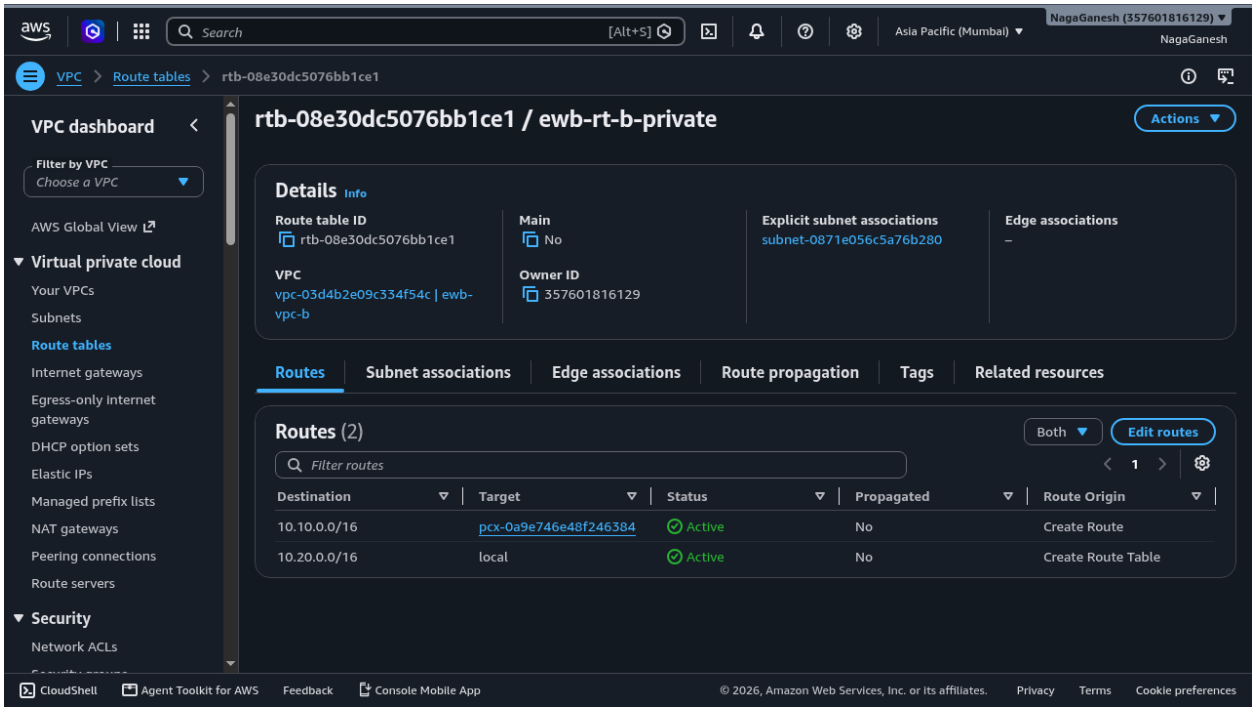
Routes: 0.0.0.0/0 to ewb-igw-a (internet), 10.10.0.0/16 local, 10.20.0.0/16 to the peering connection (added later, shown here in final state).

ewb-rt-a-private (associated: ewb-a-private)



Routes: 10.10.0.0/16 local, 10.20.0.0/16 to pcx-0a9e746e48f246384. No internet default route - App-A stays private.

ewb-rt-b-private (associated: ewb-b-private)

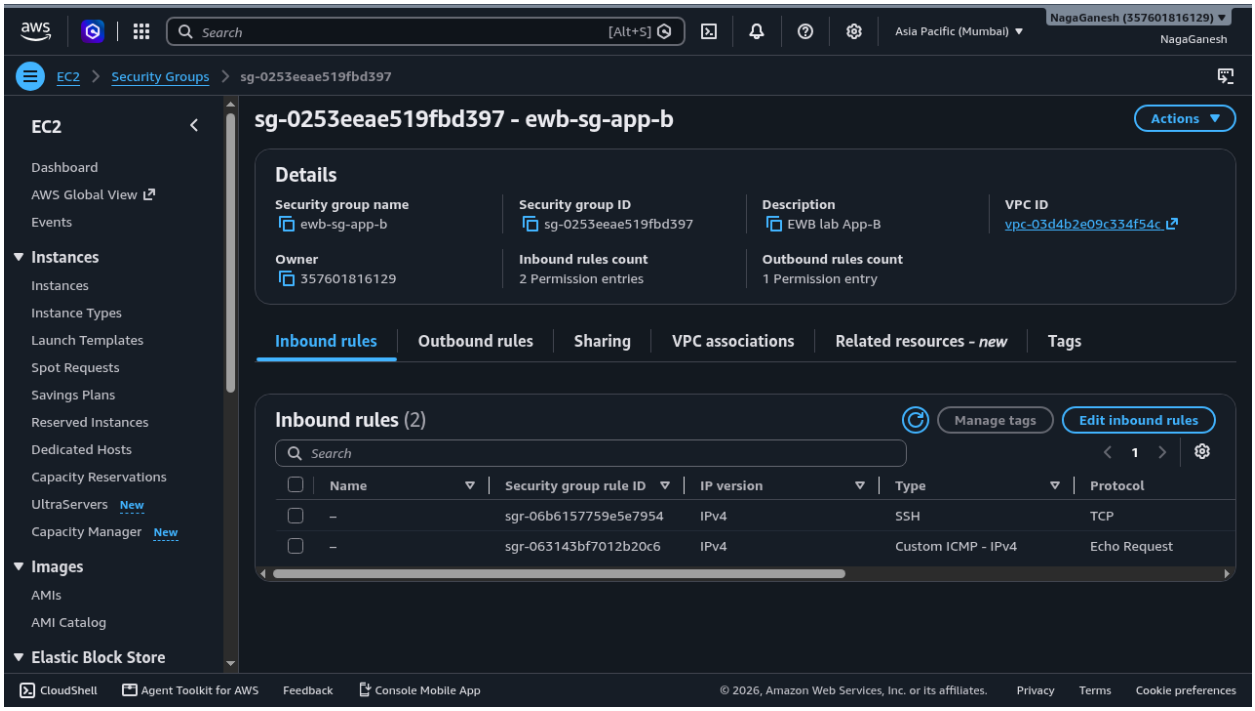


Routes: 10.20.0.0/16 local, 10.10.0.0/16 to pcx-0a9e746e48f246384 - the return path into VPC A.

4. Security groups (least-access inbound rules)

Security group	Type / port	Source
ewb-sg-bastion (VPC A)	SSH / TCP 22	8.231.155.211/32 (admin machine public IP)
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a (VPC A)	Custom ICMP IPv4 - Echo request (type 8, code 0)	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b (VPC B)	Custom ICMP IPv4 - Echo request (type 8, code 0)	10.10.2.10/32 (App-A)

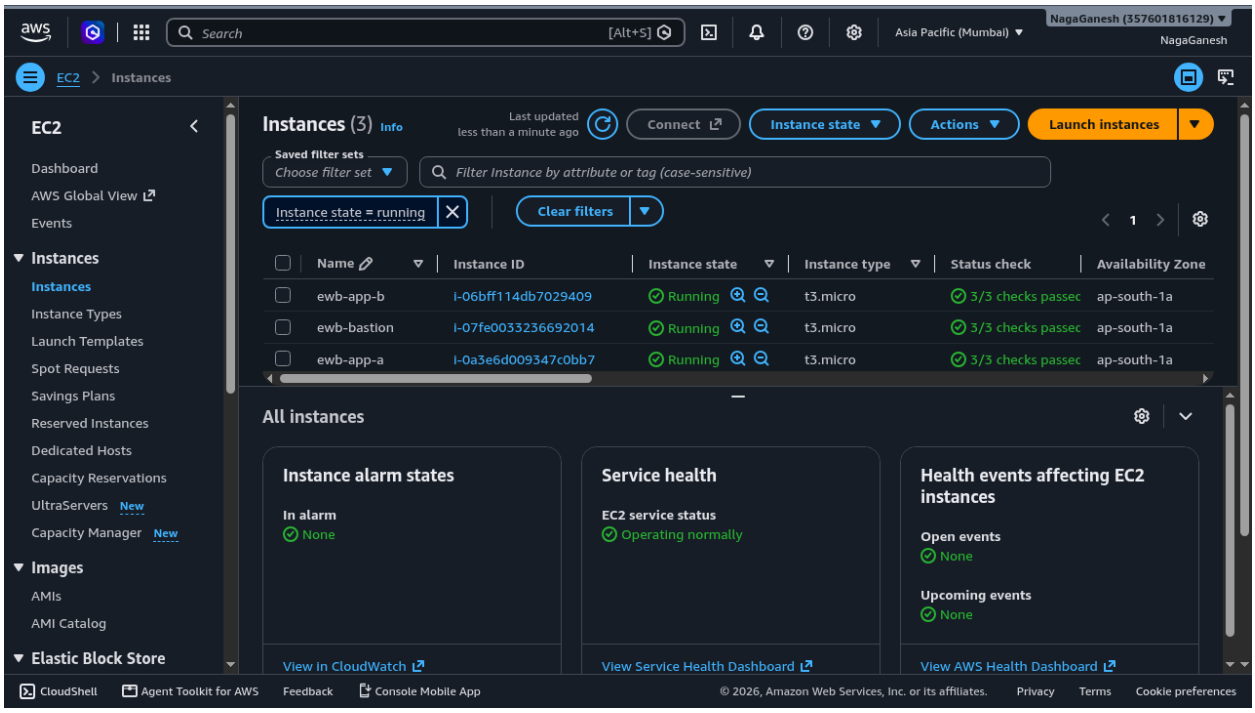
Outbound rules were left at the default allow-all. Security groups are stateful, so return traffic for allowed connections passes automatically. Default allow-all network ACLs were used. Note: every rule is a single-host /32 - nothing is open to 0.0.0.0/0.



ewb-sg-app-b inbound rules: SSH and ICMP Echo Request only (sources in the table above).

5. EC2 instances and isolation

Three t3.micro instances on Amazon Linux 2023, all using the ewb-lab key pair, launched with the fixed private IPs from the plan. Only the bastion received a public IPv4 address.



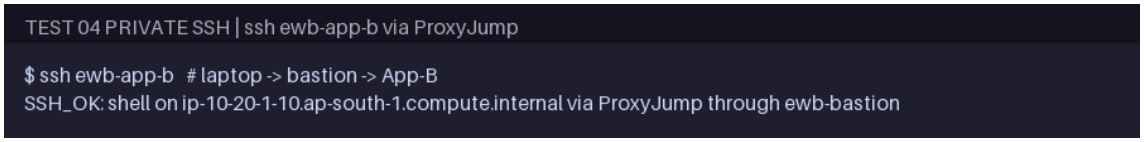
All three instances Running, 3/3 status checks passed, ap-south-1a.

Instance	Private IP	Public IPv4
ewb-bastion	10.10.1.10	13.201.3.177
ewb-app-a	10.10.2.10	none (isolated)
ewb-app-b	10.20.1.10	none (isolated)

TEST 05 - Isolation: the EC2 instance details show an empty Public IPv4 field for both App-A and App-B (CLI describe-instances also returned PublicIpAddress = None for both). The private hosts have no internet egress.

6. Administration through the bastion (SSH ProxyJump)

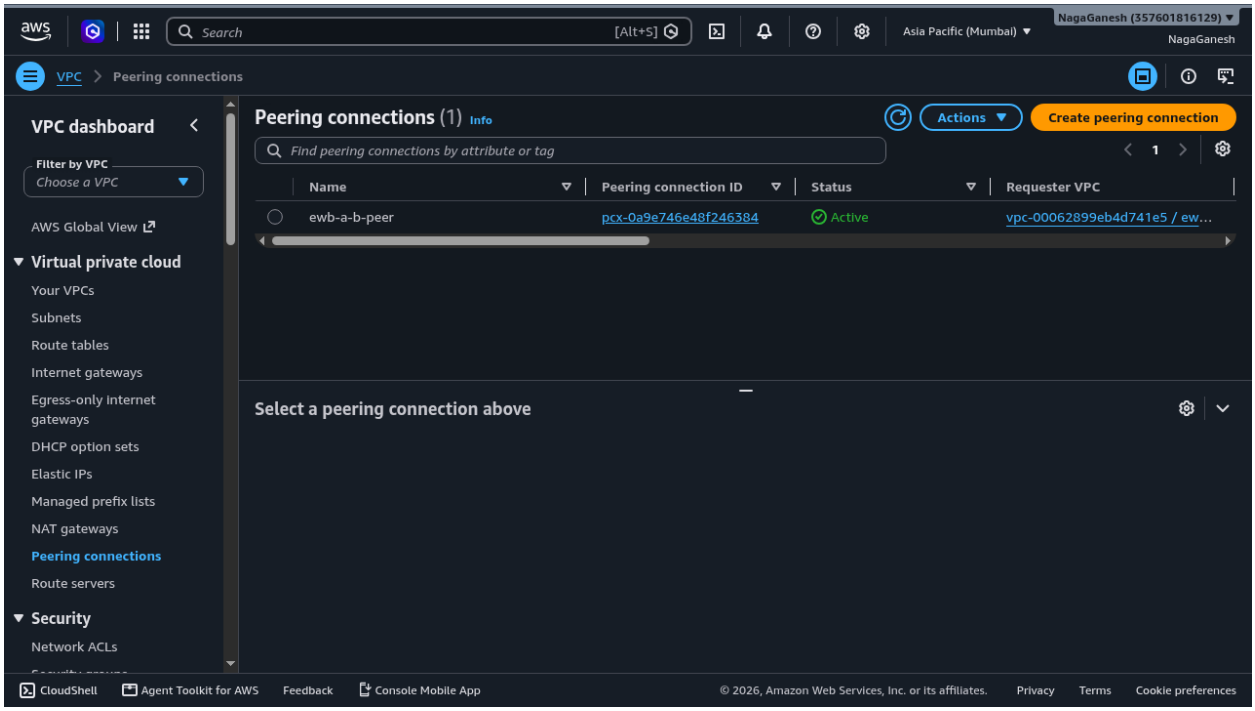
The private key stayed on the admin machine only. ~/.ssh/config defined ewb-bastion (public IP), and ewb-app-a / ewb-app-b with ProxyJump ewb-bastion. The bastion opens a second SSH connection to the target, so App-B sees the bastion's private IP (10.10.1.10) as the source - which is exactly what its SSH inbound rule allows.



TEST 04 - ssh ewb-app-b from the admin machine lands on App-B (ip-10-20-1-10) through the bastion.

7. VPC peering connection

Peering connection ewb-a-b-peer was requested from VPC A to VPC B in the same account and Region, then accepted. Status: Active. Accepting is required even between two VPCs in the same account. Peering alone does not pass traffic - the route tables on both sides (section 3) make it usable.



ewb-a-b-peer (pcx-0a9e746e48f246384) Active, requester VPC A, accepter VPC B.

8. Test evidence

TEST 01 - Baseline failure (before peering and peer routes)

From App-A, ping to App-B (10.20.1.10): 4 packets transmitted, 0 received, 100% packet loss. Expected: without peering there is no path between the VPCs.

```
TEST 01 BASELINE | ssh ewb-app-a (via bastion)

$ ssh ewb-app-a  # via ProxyJump bastion, before peering
ip-10-10-2-10.ap-south-1.compute.internal
10.10.2.10
--- baseline: ping App-B before peering ---
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3116ms

exit=1
```

TEST 02 - Forward path (after peering + routes)

App-A to App-B: 4/4 replies, 0% loss, ~0.19 ms. Path: App-A -> ewb-rt-a-private 10.20.0.0/16 route -> pcx -> App-B.

TEST 02 FORWARD | ssh ewb-app-a (via bastion)

```
$ ssh ewb-app-a   # after peering + routes
--- TEST 02 retry: App-A -> App-B ---
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.181 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.184 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.162 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.218 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3107ms
rtt min/avg/max/mdev = 0.162/0.186/0.218/0.020 ms
exit=0
```

TEST 03 - Reverse path

App-B to App-A: 4/4 replies, 0% loss. The return path works because ewb-rt-b-private has 10.10.0.0/16 to the pcx - a one-way route would not be enough, replies need the route back.

```
TEST 03 REVERSE | ssh ewb-app-b (via bastion)

$ ssh ewb-app-b   # after peering + routes
ip-10-20-1-10.ap-south-1.compute.internal
10.20.1.10
--- TEST 03: reverse path App-B -> App-A ---
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.169 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.179 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.176 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.180 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3122ms
rtt min/avg/max/mdev = 0.169/0.176/0.180/0.004 ms
exit=0
```

TEST 05a/05b - Controlled failure and recovery

Removing only App-B's inbound ICMP Echo request rule for 10.10.2.10/32 stopped the replies (100% loss) while peering and routes were untouched - proving the security rule is what permitted the traffic. Restoring the exact rule brought replies back (4/4, 0% loss). The SSH rule was never removed.

```
TEST 05a CONTROLLED FAILURE | ssh ewb-app-a

$ ssh ewb-app-a   # App-B ICMP echo rule removed
--- TEST 05a: controlled failure, ICMP rule removed on App-B ---
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3116ms

exit=1
```

```
TEST 05b RULE RESTORED | ssh ewb-app-a

$ ssh ewb-app-a   # ICMP echo rule restored
--- TEST 05b: rule restored ---
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.163 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.161 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.181 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.192 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3135ms
rtt min/avg/max/mdev = 0.161/0.174/0.192/0.012 ms
exit=0
```

9. Viva answers (be ready to explain)

Why is the return route needed? IP traffic is two-way. App-A's route table tells its subnet how to reach VPC B, but App-B's subnet needs its own route (10.10.0.0/16 to the pcx) or the echo replies have no path back - ping shows 100% loss even though the request arrives.

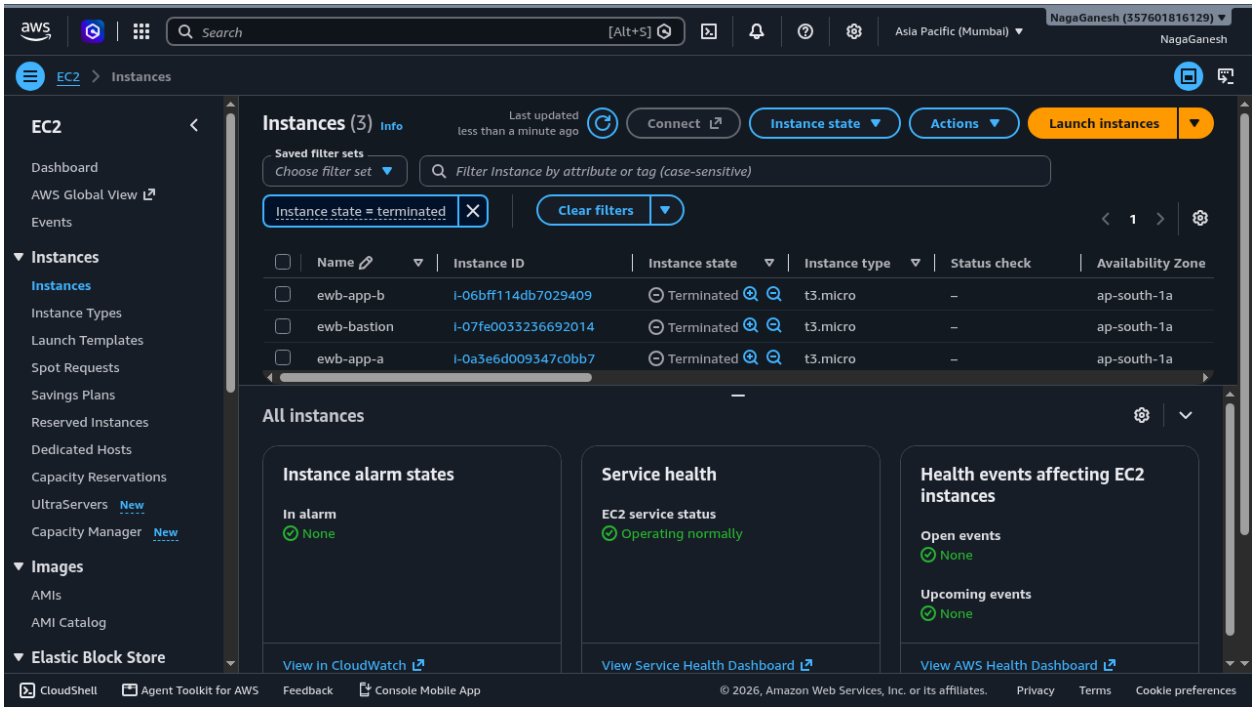
Why is the App-B SSH source the bastion private IP? ProxyJump does not forward my laptop's address. The bastion terminates my SSH session and opens a new TCP connection to 10.20.1.10 from its own interface, so the packet App-B receives has source 10.10.1.10. The security rule must allow that /32.

Why does Active peering not guarantee a successful ping? Active only means the peering link exists. Traffic still needs (a) routes on both sides pointing at the pcx, (b) a security group rule on the destination allowing the ICMP echo request, and (c) NACLs that pass it. I proved this: with Active peering, ping failed until the routes were added, and failed again when only the SG rule was removed.

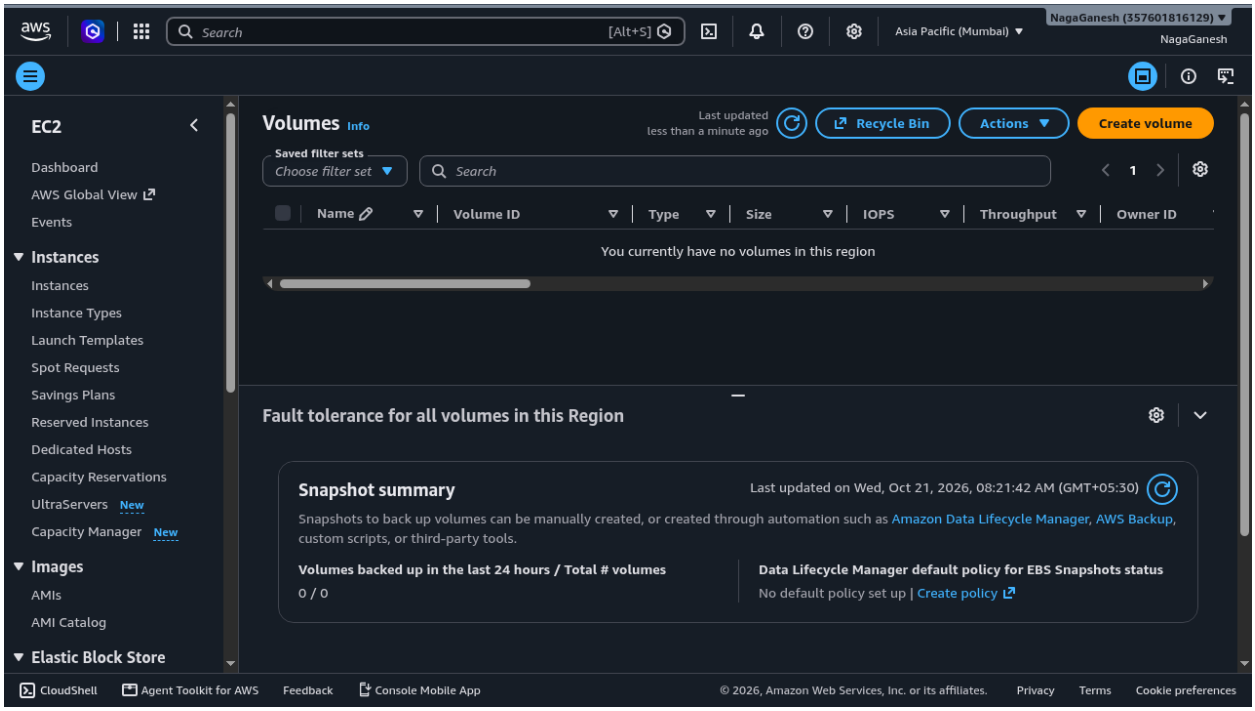
Can VPC B use VPC A's internet gateway over peering? No. VPC peering is non-transitive - VPC B cannot reach VPC A's internet gateway, and VPC A cannot act as a transit. That is by design; a NAT gateway or transit gateway would be needed for shared egress.

10. Cleanup evidence

All lab resources were removed the same day after evidence capture: instances terminated (their delete-on-termination EBS volumes removed automatically), peer routes and the peering connection deleted, security groups, subnets, route tables, internet gateway (detached then deleted), both VPCs and the ewb-lab key pair deleted, and the local .pem removed. Verification: EC2 shows the three instances Terminated and zero EBS volumes in the region; describe calls for the VPCs, subnets, route tables, security groups, IGW and key pair all return NotFound. No Elastic IPs were allocated. Nothing billable remains.



All three lab instances Terminated.



"You currently have no volumes in this region" - no leftover EBS.

11. What I learned

Peering is the link, not the permission: routes on both sides and security rules decide if traffic actually flows. I saw this twice - ping failed with Active peering but no routes, and again with routes but the SG echo rule removed. I also hit a real misconfiguration of my own: my first ICMP rules used type 8 with code 8, but an echo request is type 8 code 0, so the rule did not match and ping failed until I set the code to "all" (-1). That bug made the route-table-vs-security-group troubleshooting order from the workbook very concrete. Finally, ProxyJump changed the apparent source IP to the bastion's private address, which is why host-specific /32 rules must name the bastion, not my laptop.